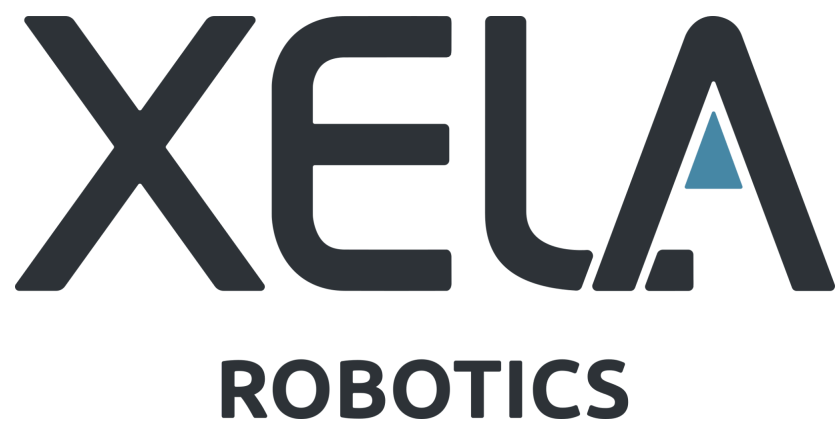




# Software Manual

---

*XELA Sensor Server*  
*Linux and Windows*  
*v1.8.0*

# Table of contents

|                                            |          |
|--------------------------------------------|----------|
| <b>General Limitations of the Software</b> | <b>4</b> |
| <b>Important notes</b>                     | <b>4</b> |
| Installation                               | 5        |
| ApplImage executables                      | 5        |
| Docker containers                          | 5        |
| Upgrade                                    | 5        |
| Major changes                              | 6        |
| <b>Using the Server</b>                    | <b>7</b> |
| Compatible CAN converters                  | 7        |
| OS compatibility                           | 9        |
| Prerequisites                              | 9        |
| Set up                                     | 10       |
| Additional First Steps for Linux           | 10       |
| Activating the CAN bus                     | 10       |
| Automating activation                      | 10       |
| Installing drivers                         | 11       |
| Windows                                    | 11       |
| Linux                                      | 11       |
| Application Installation                   | 11       |
| Windows                                    | 11       |
| Linux (ApplImages)                         | 11       |
| Linux (Docker)                             | 11       |
| Getting ready                              | 13       |
| Application specifics                      | 14       |
| XELA Configurator                          | 14       |
| XELA Server                                | 15       |
| XELA Visualisation                         | 16       |
| XELA Logger                                | 16       |
| XELA XTC                                   | 18       |
| Raspberry Pi                               | 19       |
| Compatible HATs                            | 19       |
| Application Installation                   | 19       |
| Limitations of the Raspberry Pi version    | 19       |
| How to use XELA Logger                     | 20       |
| Configuration file format                  | 20       |
| [CAN]                                      | 21       |

|                                              |           |
|----------------------------------------------|-----------|
| [server]                                     | 22        |
| CAN types                                    | 28        |
| esd                                          | 28        |
| pcan                                         | 28        |
| slcan                                        | 28        |
| socketcan                                    | 28        |
| sim                                          | 28        |
| <b>Special sensors</b>                       | <b>29</b> |
| Standard and Individually calibrated sensors | 29        |
| Prerequisites                                | 29        |
| Setup                                        | 30        |
| Notes                                        | 30        |
| Data                                         | 32        |
| How to use parameters in other software      | 32        |
| Sensors with magnetic compensation           | 33        |
| Prerequisites                                | 33        |
| Setup                                        | 33        |
| Notes                                        | 33        |
| <b>Data format</b>                           | <b>34</b> |
| Sensor Layout                                | 34        |
| Understanding Sensor Data                    | 34        |
| Determining Sensor Layout                    | 34        |
| Server Configuration                         | 34        |
| Messages                                     | 35        |
| Explanations                                 | 35        |
| <b>Example usage</b>                         | <b>37</b> |
| Code                                         | 37        |
| <b>Common errors</b>                         | <b>38</b> |
| <b>Known issues</b>                          | <b>40</b> |
| Visualisation                                | 40        |
| Server                                       | 40        |
| Configurator                                 | 40        |
| <b>How to report issues (checklist)</b>      | <b>40</b> |

## General Limitations of the Software

- The software is constantly developing and, therefore, is provided AS IS.
- The code is compiled and tested on 1 PC with Virtual Machines and might have limitations related to different hardware and software configurations.
- There might be code-breaking changes between the releases. You might need to adjust your code or settings to make your software work again. Please check the change log for details and test from a temporary directory before installing the new version!

## Important notes

- Some new Linux kernels have streamlined the CAN management, and therefore, serialCAN and PEAK devices can be pulled up with standard socketcan commands, as shown with esd devices in this manual.
- Before reporting bugs, please check the XELA support page to see if an update has already been released; if not, follow the checklist on the last page of this manual. Reports not following the checklist can take longer to investigate and fix.
- During an issue investigation, if we don't hear back for 14 days after our side's recommendation or fix, we assume the issue is resolved and close the ticket.

## Installation

You can unpack Windows and Linux executables to the directory of your liking.

**Note: The default xServ.ini file is set up for simulation.**

Before making a new configuration, the compatibility of the apps on your system can be tested by running `./xela_server -f xServ.ini` and `./xela_viz -f xServ.ini`.

## Applmage executables

Since the release of 1.7.7, the binaries are provided as Applmages for Linux (x64). Those are compiled on an Ubuntu 22.04 virtual machine with settings compatible with both Wayland and Xorg. Users are encouraged to install the Applmage files into the `/home/user/.local/bin` directory and make them executable there. To simplify their use, Applmage files are renamed to the same binary names as the native apps, and they will work across different OS versions.

## Docker containers

Since the release of 1.8.0, the application suite will also be available as a Docker image, making installation and updating easier. All our public images are available at <https://hub.docker.com/u/xelarobotics>. Please keep in mind that all images named testing will be our test builds, which we have decided to make accessible. Those contain fixes to many issues, but can also cause new bugs or major changes that will be given without documentation. We also provide an ARMv8 container compiled in Ubuntu 24.04 on Mac via Parallels.

## Upgrade

Before replacing Windows or Linux executables, try running them from a temporary directory to check the compatibility of the system's apps (See [Installation](#) for more details).

## Major changes

|                                                |                                                                                                                                                                            |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.8.0<br>(build 208601)                        | Major changes to the inner functionality, especially related to force calibration and temperature compensation<br>New and improved functionality<br>Several bug fixes      |
| 1.7.7<br>(build 161703)                        | Major fixes and optimisations<br>Improved temperature compensation                                                                                                         |
| 1.7.7<br>(build 154902)                        | Fix temperature compensation for calibrated sensors                                                                                                                        |
| 1.7.7a<br>(build 151312)                       | Add support for software-managed temperature compensation<br>Add support for the update rate of the sensor<br>Add ApplImage release to simplify the full compiling process |
| 1.7.6<br>(build 137302)                        | Fix the issue with calibrated data not working on some sensors<br>Add support for calibrated 4x6 sensors                                                                   |
| 1.7.6rc<br>(build 133105)<br>Release Candidate | Fully calibrated sensor support<br>Optional sensor modules<br>New model names<br>Upgrade to Python 3.11 on Linux (and Python 3.10 on Windows)                              |
| 1.7.5<br>(build 96513)                         | Added support for Ubuntu 22.04<br>Upgraded Python version in Windows to 3.9 with new custom ntcan library for Windows and Ubuntu 20.04 and 22.04<br>Bug fixes              |



### Warning

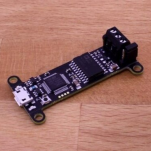
Version 1.8.0 will support newer .xela files that allow us to update algorithms by changing only the core module file (uSPa44 and others) instead of all individually calibrated files. Those functions are not available for older software, but the files themselves might work fine.

# Using the Server

## Compatible CAN converters

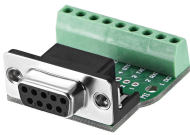
Note: The following list includes only devices that we have tested. Other devices might also work. Please see [xela.lat-d5.com/canusb](http://xela.lat-d5.com/canusb) for more details and an up-to-date list.

| Converter                                                                           | Name/model                     | Overview                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | Esd CAN-USB/2<br>Esd CAN-USB/3 | Works in Windows (esd)<br>Works in Linux (socketcan)<br><br>esd CAN-USB/3 FD is fully supported in Windows and is in the kernel from 6.6 onwards for Linux<br><br>Not supported in Windows for ARM |
|   | PEAK USB-CAN                   | Works in all Windows and Linux releases (pcan)<br><br>Drivers and API are limited and can occasionally cause delays and other issues                                                               |
|  | LAWICEL<br>CANUSB              | Linux-only (socketcan)<br><br>Uses Serial-over-USB to communicate, and can end up with a bottleneck at heavy sensor coverage                                                                       |
|  | VScam<br>USB-CAN Plus          | Linux-only (socketcan)<br><br>Works with Raspberry Pi as well                                                                                                                                      |
|  | InnoMaker<br>CAN-USB           | Linux-only (socketcan)<br><br>Works reliably for its low price                                                                                                                                     |
|  | Kvaser Leaf v3                 | Works in Linux (socketcan)<br>Windows will be added in the 1.8.1 update<br><br>Suitable for industrial use due to its reliability                                                                  |

| Converter                                                                         | Name/model                                             | Overview                                               |
|-----------------------------------------------------------------------------------|--------------------------------------------------------|--------------------------------------------------------|
|  | CANable (Pro)                                          | Works in Linux with “candlelight” firmware (socketcan) |
|  | 2-Channel<br>CAN-BUS(FD)<br>Shield for<br>Raspberry Pi | Works with Raspberry Pi (socketcan)<br><br>Has 2 buses |
|  | Waveshare<br>RS485 CAN HAT                             | Works with Raspberry Pi (socketcan)                    |

If using a CAN converter without a DB9 connector, you can buy a DB9 breakout board online.

Connect the cables between the converter and DB9 breakout board as follows:



CAN\_H → Pin 7  
CAN\_L → Pin 2  
GND → Pin 3 (highly recommended)



### Warning

Even though other CAN devices may work, a misconnection could damage the sensor.

Please always use CAN devices with the standardised DB-9 connector.

## OS compatibility

- Windows 11 (64-bit with [Visual C++ redistributable 2015-2019 x64](#) installed)
- Ubuntu 22.04+ (64-bit)  
Other 64-bit Linux flavours, i.e. Debian with kernel 5.13 or newer
- ARM64 v8 Linux devices - use only through a Docker container
- ARM v7 Linux devices (Raspberry Pi 4B) - use only through a Docker container

Note: Other Windows and Linux releases may work, but they are not tested.

Note: Docker containers will be released separately from the native software.

## Prerequisites

For connecting to the XELA Server with a user-made Python script, the following Python packages would need to be installed:

- websocket-client

For other languages, compatible libraries would be required.



### Warning

Please ensure that sensors receive sufficient power. An external USB power supply is recommended.

**There are 2 USB cables for sensors. Both MUST be connected**

Visualisation can be very CPU-intensive; please use a high-performance PC with a dedicated GPU or a powerful CPU (i7+/Ryzen 7+) to run it, or consider visualisation on another computer.

## Set up

### Additional First Steps for Linux

#### Activating the CAN bus

**Linux only** (Windows requires only drivers. Windows users can skip this part)

If you haven't activated CAN bus yet, please run the following commands:

**Note:** Make sure you have [can-utils](#) installed first

- **Esd or any other CAN device (includes serialCAN and PEAK with the newer kernel):**  
`user@localhost:~$ sudo ip link set up can0 type can bitrate 1000000`  
Explanation: First, we will set up the CAN network on can0 with a bitrate of 1Mbit/s and pull it up
- **Any serial CAN (slcan) device:**  
`user@localhost:~$ sudo slcand -o -s8 -t hw -S 3000000 /dev/ttyUSB0`  
`user@localhost:~$ sudo ifconfig slcan0 up`  
Explanation: First, we set up an SLCAN device running on /dev/ttyUSB0 with a hardware bitrate of 3Mbit/s and CAN speed of 1Mbit/s (-s8), then we will pull it up

**NOTE:** Replace can0/slcan0 with the corresponding CAN bus name and ttyUSB0 with the correct device name

**NOTE:** On some Ubuntu 22.04 and newer installations, the slcand has been upgraded, and it will identify the network as can0 instead of slcan0 so that you can use the same commands as for ESD

#### Automating activation

It is possible to set up an automated pull-up of the CAN buses in Linux. For that, it is recommended to check the examples at [github.com/mcsix/can-device-rules](https://github.com/mcsix/can-device-rules)

If you encounter issues or have alternatives, feel free to open an issue there for discussion.

Those will require a more profound understanding of the Linux operating system, and if handled incorrectly, can cause the system to break.

## Installing drivers

### Windows

Please install the drivers for your CAN-USB converter by downloading them from the manufacturer's website.

### Linux

Most drivers are already included in the kernel; however, for ESD USB-CAN/3 FD support in kernels prior to 6.6, you may need to request the kernel patch from ESD or contact us for detailed instructions.

## Application Installation

### Windows

Unpack the downloaded files to your preferred folder

Keep all configuration files in the same folder.

**Important:** Make sure you have [Visual C++ redistributable 2015-2019 x64](https://aka.ms/vs/16/release/vc_redist.x64.exe) installed from [https://aka.ms/vs/16/release/vc\\_redist.x64.exe](https://aka.ms/vs/16/release/vc_redist.x64.exe)

### Linux (ApplImages)

Unpack the downloaded zip file to your preferred directory (we would recommend adding that directory to PATH, especially if you plan to use ROS)

Make /etc/xela directory and give it 777 permissions (can be done with the xela\_conf application as well)

Keep all configuration files in the /etc/xela directory.

### Linux (Docker)

Some platforms will be listed only. Please check <https://hub.docker.com/u/xelarobotics> for more details. For launching and controls, you can use the following bash script for ARM64 v8:

```
#!/bin/bash

# Check if an argument is provided
if [ -z "$1" ]; then
    echo "Error: No application specified."
    exit 1
fi

# Define allowed applications (with the xela_ prefix)
allowed_apps=("xela_viz" "xela_server" "xela_conf" "xela_log")

# Add the "xela_" prefix only if not already present
if [[ ! "$1" =~ ^xela_ ]]; then
    app_name="xela_$1"
else
    app_name="$1"
fi

# Validate application
if [[ ! " ${allowed_apps[@]} " =~ " ${app_name} " ]]; then
    echo "Error: Invalid application specified. Allowed applications: ${allowed_apps[@]}"
    exit 1
fi

# Allow container access to the X server
xhost +local:

# Check if running in Docker rootless mode by looking for DOCKER_ROOTLESS_ROOT?
if [ -n "$DOCKER_ROOTLESS_ROOT" ]; then
    # In rootless mode, host networking is not available.
    # Define explicit port mapping (adjust as necessary)
    network_options="-p 5000:5000"
else
    # Non-rootless, use host networking if desired.
    network_options="--network host"
fi

# Run the container with adapted options
docker run --rm -it \
    ${network_options} \
    -v /etc/xela:/etc/xela \
    -e SDL_VIDEODRIVER=x11 \
    -e LIBGL_ALWAYS_SOFTWARE=1 \
    -e DISPLAY=$DISPLAY \
    -e XDG_RUNTIME_DIR=$XDG_RUNTIME_DIR \
    -v /tmp/.X11-unix:/tmp/.X11-unix \
    -v $XDG_RUNTIME_DIR:$XDG_RUNTIME_DIR \
    xelarobotics/xela_suite_arm_testing:latest "$app_name" "${@:2}"

# Revoke X server access after the container exits
xhost -local:
```

If you need to use a different local directory for the files, specify it on the line about /etc/xela, as the applications will internally use /etc/xela for logs, xServ.ini, and sensor files.

## Getting ready

1. Make sure all cables are connected. There are 2 USB cables for CAN devices, and both MUST be connected, as the one on the sensor side of the CAN-USB converter is for power.
2. Unpack the files to a suitable location.  
In the case of the ROS package, in your workspace folder
3. Open your terminal window and navigate to the correct directory where the executables are
4. Set up the xServ.ini file, either by running the configurator (esd and pcan only in Windows, any socketcan in Linux; 4x4 and 4x6 sensors only, more support in 1.8.1) or by editing the file manually based on the details provided in [Configuration file format](#)  
**NOTE:** If using a CAN adaptor other than the esd CAN-USB/2 or that is not added to the kernel in Linux, you will need to specify it with the -c flag (i.e. ./xela\_conf -c slcan0)

Windows PowerShell: ./xela\_conf.exe

Windows cmd: xela\_conf.exe

Linux: ./xela\_conf

Optional: You may also want to run XELA\_XTC for all sensors before use to ensure the best temperature compensation. Please see more in [XTC](#)

5. Run the server (mandatory)

The server will send the data via Websockets. See the arguments section to see all configuration options.

Windows PowerShell: ./xela\_server.exe

Windows cmd: xela\_server.exe

Linux: ./xela\_server

6. Run additional tools

- Visualizer for showing the sensor status on the screen as dots (size and location based on data)

Windows PowerShell: ./xela\_viz.exe

Windows cmd: xela\_viz.exe

Linux: ./xela\_viz

- Logger to log sensor data in raw format with UNIX timestamps into the LOG folder as CSV files (all sensors will have a separate filename in the format “sens-[sensor number].csv”. Please see the arguments section to see more detailed information about using it

Windows PowerShell: ./xela\_log.exe

Windows cmd: xela\_log.exe

Linux: ./xela\_log

## Application specifics

### XELA Configurator

xela\_conf

```
[XELA]           We found 2 sensors

Sensor 1:
  Model: uSPa44 (XR2244)
  ID: 1
  Size: 4x4
  Starting SDA: 0

Sensor 2:
  Model: uSPa46 (XR1946)
  ID: 11
  Size: 4x6
  Starting SDA: 0

If all sensor data is correct, please enter y and press Enter to save the configuration. Save: █
```

XELA Configurator (Sensor AutoConfig) is a tool that automatically sets up the xServ.ini. It expects the esd CAN-USB/2 device to be used if no arguments are given. When executing, the user must specify the device type and channel in the terminal for all other devices, or if the bus (e.g., can0 in Linux) or net ID (e.g., 0 in Windows) is different.

**NOTE:** Only for **uSPa 44** and **uSPa 46** sensors, more support in 1.8.1

#### Windows examples:

|                                         |                                                                                                |
|-----------------------------------------|------------------------------------------------------------------------------------------------|
| ./XELA_Conf.exe -d esd -c 0             | Default, esd CAN-USB/2 device on Net ID 0                                                      |
| ./XELA_Conf.exe -d slcan -c COM3        | VScom or other serial-can devices that show up on COM3<br>Note: Serial connection can be buggy |
| ./XELA_Conf.exe -d pcan -c PCAN_USBBUS1 | PEAK CAN-USB devices, bus/channel names can differ                                             |

#### Linux examples:

|                                     |                                                             |
|-------------------------------------|-------------------------------------------------------------|
| ./xela_conf -d socketcan -c can0    | Default, esd CAN-USB/2 device on can0                       |
| ./xela_conf -d socketcan -c slcan0  | VScom or other serial-can devices that are set up on slcan0 |
| ./xela_conf -d pcan -c PCAN_USBBUS1 | PEAK CAN-USB devices, bus/channel names can differ          |

## XELA Server

xela\_server

```
[SENSORS]      Sensor 1
ID: 11
Size: 4x4
Model: uSPa44
File: __internal__/XR2244.xela
Starting SDA: 0
Rotation: 0°
Description: Standard 4x4 sensor
Modules for uSPa44:
    Module 1: Raw Data of Magnetic Compensation (rawdata) L-mode Status: OFF (default)
    Module 2: Magnetic Compensation (magcomp) L-mode Status: OFF (default)
    Module 3: Force Calibration (calibration) C-mode Status: OFF (default)
    Module 4: Clustering (clustering) C-mode Status: OFF (default)

Layout:
  03b 0  02b 0  01b 0  00b 0
  13b 0  12b 0  11b 0  10b 0
  23b 0  22b 0  21b 0  20b 0
  33b 0  32b 0  31b 0  30b 0
```

XELA Server is the primary tool that connects the sensors to other software.

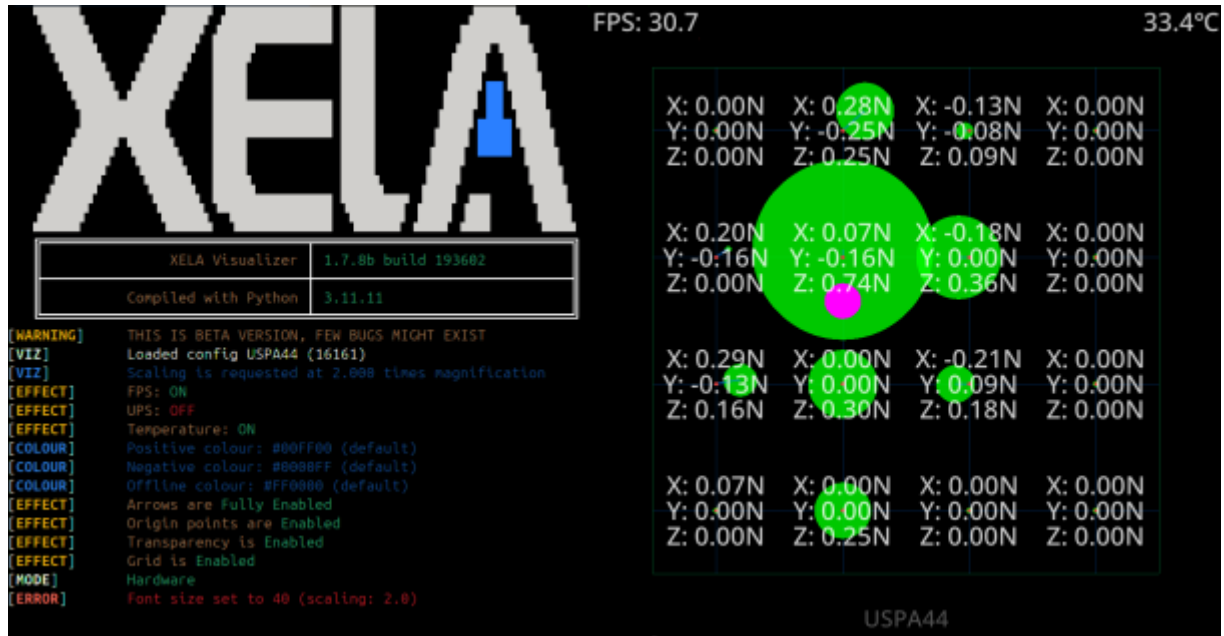
It will require an xServ.ini file to be present or an alternative specified with arguments.

Some of the essential arguments:

|                      |                                                                                                                                                                          |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -f filename          | To specify the alternative for xServ.ini                                                                                                                                 |
| --ip xxx.xxx.xxx.xxx | To specify the device's IP address to be accessible only over the specified IP. The default is 0.0.0.0, which is accessible for both localhost and external connections. |
| --port x             | Specifying a port other than 5000 enables the use of multiple server apps on the same computer.                                                                          |

## XELA Visualisation

xela\_viz



XELA Visualisation is the tool used to visualise the sensor's activity.

It requires both the Server to be running and the xServ.ini to exist.

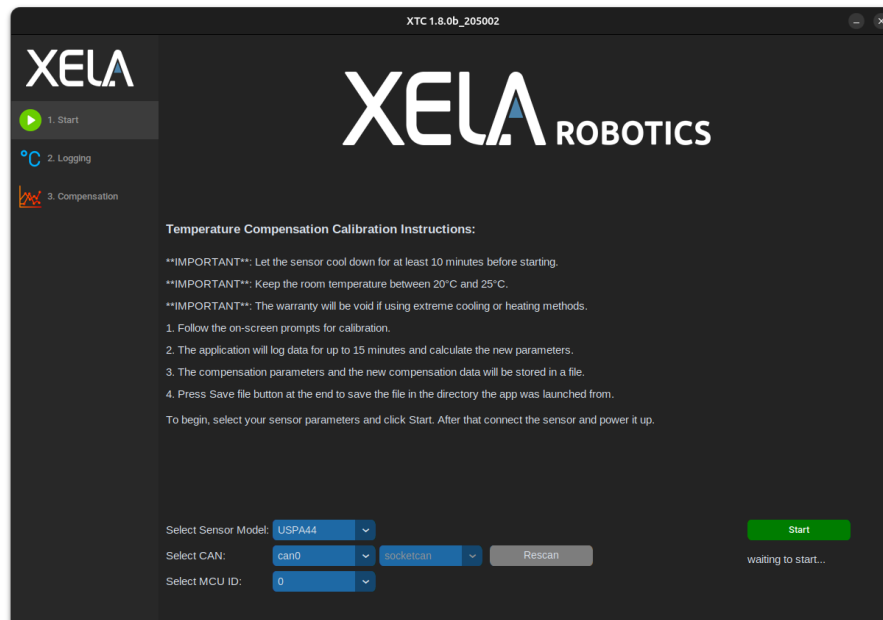
Some important arguments:

|                      |                                                                                                                                               |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| -f filename          | To specify the alternative for xServ.ini                                                                                                      |
| --ip xxx.xxx.xxx.xxx | To specify the IP of the server if the Server is running on a different computer or is not available for localhost                            |
| --port x             | To specify the port of the Server or middleware if it is not 5000                                                                             |
| --viz x              | To specify the sensor to visualise when more than one is available on the Server, and you are not trying to visualise the first one.          |
| --bright             | To run the visualisation with a white background. You may need to change the colours of the taxels in the INI file to make them more visible. |



## XELA XTC

xela\_xtc



XELA XTC is a new tool that can determine the most accurate temperature compensation parameters for each sensor. This tool will be part of our suite, allowing people to update their parameters. By adjusting those parameters, the server application can compensate for temperature drifts and provide more accurate force measurements, even after prolonged use. It is purely a GUI-based app to make it easier to use.

**Steps**

1. The user unpowers the bus and disconnects the sensor, leaving CAN active.
2. The user will let the sensor cool down (up to 15 minutes).
3. The user sets up the configuration in the app and presses Start.
4. Once the sensor is cooled down, the user connects it to the CAN cable and re-powers the bus.
5. The app will record the data (up to 15 minutes) and find the most optimal parameters.
6. At the end, the user can view the graphs of the improvements and decide if they want to override the standard parameters. If they do, the user must press Save, after which an .xtp file will be created. The user needs to move this file to the app directory (or /etc/xela/sensors/ directory) and add the filename (without extension) to the xServ.ini file under [sensorN] by specifying temp\_id = given\_filename.

Example:

temp\_id = USPA44\_1739412210

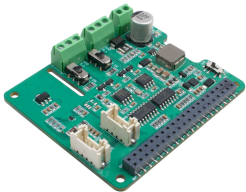
**Warning:** If you use Windows and have set the ESD device NET ID above 15, this application will not work

## Raspberry Pi/SBC

**Note:** As of 1.8.0, ARM-based releases will be available only through Docker containers.

**Note:** Docker containers will only have occasional releases and, therefore, might be several versions behind the main applications

### Compatible HATs

|                                                                                    |                                                                                   |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|   | Sseed Studio 2-Channel CAN-BUS(FD) Shield for Raspberry Pi<br>Pro: 2 CAN channels |
|  | WaveShare RS485 CAN HAT                                                           |

### Application Installation

Applications can be installed in any suitable directory. With Docker containers, the user must define to directory that would be mounted as /etc/xela and include the xServ.ini file and any additional files necessary, like sensor .xela and temperature .xtp files.

### Limitations of the SBC version

The version for single-board computers may support fewer CAN-USB devices, as some may have been stripped from the kernel.

## How to use XELA Logger

Entries must use one of the following formats:

| Data          | Meaning                                                                                                    |
|---------------|------------------------------------------------------------------------------------------------------------|
| 1             | Sensor* 1 will be read (default 100 broadcast cycles*)                                                     |
| 1,2,3         | Sensors 1, 2 and 3 will be read (default: 100 broadcast cycles)                                            |
| 1-3           | Sensors 1,2, and 3 will be read (default: 100 broadcast cycles) (shortcut)                                 |
| 1:150         | Sensor 1 will be read 150 broadcast cycles                                                                 |
| 1:150,2       | Sensor 1 will be read 150 broadcast cycles, and sensor 2 for a default of 100                              |
| 1-3:150       | Sensors 1, 2 and 3 will all be read for 150 broadcast cycles                                               |
| 1-5,2:150     | Sensor 2 will be read 150 broadcast cycles, and sensors 1, 3, 4 and 5 for default 100                      |
| 1:60t         | Sensor 1 will be read for 60 seconds                                                                       |
| 1-3:30t       | Sensors 1, 2 and 3 will all be read for 30 seconds                                                         |
| 1-5,2:30t     | Sensor 2 will be read for 30 seconds, and sensors 1, 3, 4 and 5 for a default of 100 broadcast cycles      |
| 1-5:200,3:30t | Sensor 3 will be read for 30 seconds, and sensors 1, 2, 4 and 5 will all be read for 200 broadcast cycles. |

Example:

|   | A                | B     | C     | D     | E     | F     | G     |
|---|------------------|-------|-------|-------|-------|-------|-------|
| 1 | time             | 1X    | 1Y    | 1Z    | 2X    | 2Y    | 2Z    |
| 2 | 1587029110.17588 | 16858 | 18758 | 35645 | 15128 | 15545 | 31502 |
| 3 | 1587029110.17615 | 16858 | 18758 | 35645 | 15128 | 15545 | 31502 |
| 4 | 1587029110.27562 | 19188 | 16018 | 35189 | 17276 | 15273 | 32799 |
| 5 | 1587029110.2757  | 19188 | 16018 | 35189 | 17276 | 15273 | 32799 |
| 6 | 1587029110.37499 | 13737 | 14693 | 34553 | 18699 | 16100 | 32949 |
| 7 | 1587029110.37511 | 13737 | 14693 | 34553 | 18699 | 16100 | 32949 |

### Note

\*When defining an argument for the -s flag, ensure that there are NO spaces in it (after a comma or a colon)

\*If the sensor is defined multiple times, the latest entry counts

\*Sensors can be from 1 to 64 (to 16 before 1.7.1)

\*Time is recorded as a **Unix timestamp** (on all OSs).

The **Unix epoch** (or **Unix time** or **POSIX time** or **Unix timestamp**) is the number of seconds that have elapsed since January 1, 1970 (midnight UTC/GMT), not counting leap seconds (in ISO 8601: 1970-01-01T00:00:00Z). The epoch is Unix time 0 (midnight 1/1/1970), but 'epoch' is often used as a synonym for Unix time. Some systems store epoch dates as a signed 32-bit integer, which might cause problems on January 19, 2038 (known as the Year 2038 problem or Y2038).

\*Broadcast cycles depend on the computer configuration and are defined as periodic update cycles

\*If supported, the force measurements will be added to this file

## Configuration file format

If your INI file requires a custom format, follow the specific instructions below.

Sections in the config file:

**[CAN]** – defines settings for CAN bus (**will be in [server] from 1.8.0 onwards**)

**[server]** – defines settings for the server

**[viz]** – defines settings for the visualisation

**[sensor]** – defines primary sensor settings and the number of total boards

**[sensorN]** – (*N* is an integer at least 1, sequential) – defines Nth sensor's settings; if different from defaults, you can define *[sensor1]* if it will be different from default settings

Alternative names for the element are in grey.

| Section/element                        | notes/description                                                                                                                                                                                                                                  |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>[CAN]</b>                           | Note: This will be deprecated, and all elements will be moved to [server] in 1.8.1                                                                                                                                                                 |
| bustype = socketcan<br>busmode<br>type | Bus types based on the driver:<br>Linux: <b>socketcan</b> , <b>pcan</b><br>Windows: <b>esd</b> , <b>pcan</b> , <b>slcan</b><br>Testing: <b>sim</b>                                                                                                 |
| channel = can0<br>can                  | Channel used for CAN communication, examples:<br>pcan: <b>PCAN_USBBUS1</b><br>esd: <b>0</b><br>socketcan: <b>can0</b> / <b>slcan0</b><br>With multi-CAN, the channels must be separated by commas, without spaces. I.e. <i>channel = can0,can1</i> |

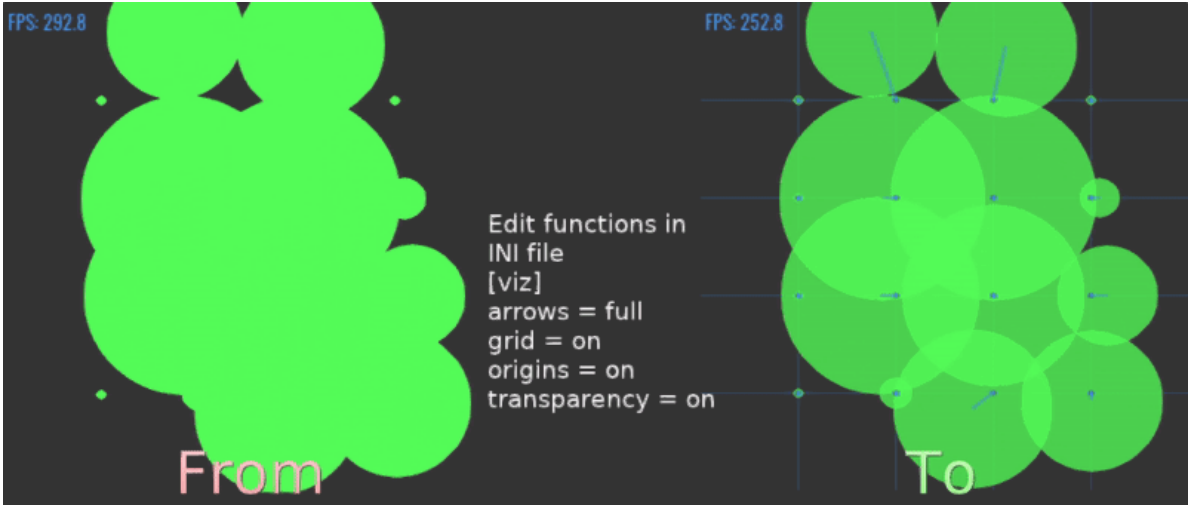
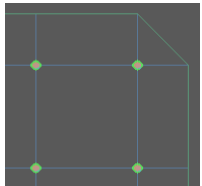
Format details continue on the next page.

Format details continued

| Section/element                        | notes/description                                                                                                                                                                                                  |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>[server]</b>                        | Bustype and channel are supported here since 1.7.6; they will overwrite details given in [CAN]                                                                                                                     |
| bustype = socketcan<br>busmode<br>type | Bus types based on the driver:<br>Linux: <b>socketcan</b> , <b>pcan</b><br>Windows: <b>esd</b> , <b>pcan</b> , <b>slcan</b> , <b>kvaser</b><br>Testing: <b>sim</b>                                                 |
| refreshrate = 500                      | Set a periodic send rate per second for the server<br>The default is up to 500 Hz, but it can be limited.<br>Must be an integer between 1 and 2000.<br>In the event of sensor delays, please reduce this settingw. |

Format details continue on the next page.

## Format details continued

| Section/element                                                                    | notes/description                                                                                                                                                                                                                                                  |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [viz]                                                                              | Specific settings only for XELA Visualizer                                                                                                                                                                                                                         |
|  |                                                                                                                                                                                                                                                                    |
| max_offset = 100                                                                   | Defines how far the “bubbles” will go in visualisation<br>100 is the gap between 2 taxels (bubbles) in idle                                                                                                                                                        |
| max_size = 100                                                                     | Defines the maximum size of the “bubbles” in the visualisation<br>100 is the gap between 2 taxels (bubbles) in idle                                                                                                                                                |
| arrows = full                                                                      | Defines if arrows are shown:<br><b>full</b> - show from the origin to the centre of the new position<br><b>half</b> - show from the origin up to halfway to the centre of the new position<br><b>none</b> - hide all lines between the origin and the new position |
| grid = on                                                                          | Defines if the grid of the sensor is shown:<br><b>on</b> - shown<br><b>off</b> - hidden<br><i>Some sensors will have a custom grid that helps to visualise them better</i>    |
| origins = on                                                                       | Defines if origin points are shown:<br><b>on</b> - shown<br><b>off</b> - hidden                                                                                                                                                                                    |
| transparency = on                                                                  | Defines if taxels have transparency:<br><b>on</b> - shown<br><b>off</b> - hidden<br><b>Note:</b> It can be taxing on the CPU without a dedicated GPU                                                                                                               |

Format details continue on the next page.

Format details (for [viz]) continued

| Section/element                       | notes/description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| fps = on                              | <p>Defines if the visualisation window shows the FPS counter for its redraw rate</p> <p>In HW mode, it will try to match the screen refresh rate</p> <p>In SW mode, it will show the total maximum redraw rate</p> <p>The default mode is HW since 1.7.7</p> <p>Supported values for this setting:</p> <p><b>on</b> - shown</p> <p><b>off</b> - hidden (default)</p> <p><b>Note:</b> can also be defined under [sensor] and [sensorM] sections</p> <p><b>Note:</b> should only be enabled if visualisation performance is low and running in software mode has no change</p> |
| ups = on                              | <p>Defines if the visualisation window shows a UPS counter for sensor update rate</p> <p>It will only be shown if the server broadcasts the information</p> <p>Supported values for this setting:</p> <p><b>on</b> - shown</p> <p><b>off</b> - hidden (default)</p> <p><b>Note:</b> can also be defined under [sensor] and [sensorM] sections</p>                                                                                                                                                                                                                            |
| temp_show = on                        | <p>Defines if temperature info is shown</p> <p><b>on</b> - shown (default)</p> <p><b>off</b> - hidden</p> <p><b>Note:</b> will only be shown if supported by the sensor and server</p>                                                                                                                                                                                                                                                                                                                                                                                       |
| color_pos,<br>color_neg,<br>color_off | <p>Those 3 allow overriding the default “bubble” colours.</p> <p>Colours are in RGB HEX format without the leading #</p> <p>Defaults are:</p> <p>color_pos = 00FF00</p> <p>color_neg = 0000FF</p> <p>color_off = FF0000</p> <p>In case of transparency, the visualiser will automatically apply it, and that is not customisable</p> <p>You can also turn off the specific colour by defining it as 000000</p>                                                                                                                                                               |

Format details continue on the next page.

## Format details continued

| Section/element                                                            | notes/description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>[sensor]</b>                                                            | Details about the default sensor (unless overridden in [sensorM])                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| num_of_sensor = 2<br>num_brd<br>sensors                                    | Defines the number of sensors to be broadcast by the server.<br>If using multiple small (1x1, 2x1, 2x2) sensors on a single controller, the sensors different from the first must specify it, i.e. <code>channel=2</code>                                                                                                                                                                                                                                                                                                                                                 |
| version = 3<br>ctrl_ver<br>controller_ver<br>controller_version<br>ctr_ver | Defines the controller version<br>By default (if not specified), it will be 3.<br>Version 1 was supplied in 2018 - early 2019 only.<br>Versions 2 and 3 have different X, Y, and Z resolutions (15/16 bit HW)                                                                                                                                                                                                                                                                                                                                                             |
| model = XR1944<br>mdl                                                      | Defines the sensor model (or serial number)<br>The default ones are:<br><b>XR1844</b> - old 4-channel 16-taxel sensor (no longer sold)<br><b>XR1944</b> - new 2-channel 16-taxel sensor (no longer sold)<br><b>uSPa11</b> - 1x1 sensor (formerly XR1911)<br><b>uSPa21</b> - 2x1 sensor (formerly XR1921)<br><b>uSPa22</b> - 2x2 sensor (formerly XR1922)<br><b>uSPa46</b> - 4x6 sensor (formerly XR1946)<br><b>uSPa44</b> - 4x4 sensor (formerly XR2144 and XR2244 with additional features)<br><b>Individually calibrated sensors</b> - will have the serial number here |
| id = 6<br>ctr_id<br>controller_id<br>ctrl_id                               | Defines the controller ID<br>It must be written in decimals, such as numbers 0 to 15.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| channel = 0<br>chan<br>sda                                                 | Defines the starting channel (SDA) for the sensor                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| taxel = 0<br>txl                                                           | Defines the taxel ID offset<br>It must be written in decimals, such as numbers 0 to 15.<br>Used with some sensors, primarily uSPa 11/21/22 sensors                                                                                                                                                                                                                                                                                                                                                                                                                        |

Format details continue on the next page.

Format details continued, section [sensor]

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rotation = 0<br>rot      | Defines how much the sensor will be rotated for adjustment<br><b>0</b> - no rotation<br><b>1</b> - 90 degrees clockwise<br><b>2</b> - 180 degrees<br><b>3</b> - 90 degrees anti-clockwise<br><b>NOTE: Some modules might not work with rotation</b>                                                                                                                                                                                                           |
| <u>module_name</u> = off | Enable or disable custom modules supported by the sensor model.<br>Most modules are off by default.<br><b>off</b> - force the module to be disabled<br><b>on</b> - force the module to be enabled<br><br>Example modules:<br><b>magcomp</b> - magnetic compensation<br><b>clustering</b> - calculated touch clusters<br><b>calibration</b> - force calibration<br><br><b>Warning:</b> Enabling modules not supported by your hardware can cause invalid data. |
| temp_id = XYZABC         | To load temperature compensation parameters for the sensor.<br>XYZABC stands for the filename without the .xtp extension.<br><a href="#">XELA XTC</a> is a tool to make those files                                                                                                                                                                                                                                                                           |

Format details continue on the next page.

## Format details continued

| Section/element     | notes/description                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>[sensorM]</b>    | Contains only details that override the default sensor; check alternative naming from [sensor]                                                                                                                                                                                                                                                                                                                                                                          |
| version = 3         | Defines the controller version                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| model = uSPa44      | Defines the sensor model                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| id = 6              | Defines the controller ID                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| taxel = 0           | Defines the taxel offset                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| channel = 0         | Defines the starting channel (SDA) for the sensor                                                                                                                                                                                                                                                                                                                                                                                                                       |
| rotation = 0        | Defines how much the sensor will be rotated for adjustment<br><b>0</b> - no rotation<br><b>1</b> - 90 degrees clockwise<br><b>2</b> - 180 degrees<br><b>3</b> - 90 degrees anti-clockwise                                                                                                                                                                                                                                                                               |
| <i>module</i> = off | Enable or disable custom modules supported by the sensor model.<br>Most modules are off by default.<br><b>off</b> - force the module to be disabled<br><b>on</b> - force the module to be enabled<br><br>Example modules:<br><b>magcomp</b> - magnetic compensation<br><b>clustering</b> - calculated touch clusters<br><b>calibration</b> - force calibration<br>Warning: Enabling modules that are not supported by your hardware can produce invalid resultant data. |

## CAN types

### esd

By default, the esd CAN-USB/2 uses NTCAN drivers provided by esd. As Windows lacks support for standardised CAN, the user is required to install the latest drivers and SDK found on the [esd.eu](http://esd.eu) website.

Channels or net IDs are numeric and can be defined in Device Manager to support multiple devices on the same PC.

### pcan

Most PEAK devices use a non-standardised communication protocol between the PC and CAN converter. It works both in Windows and Linux.

Channels are named based on the device type and sequence, i.e. PCAN\_USBBUS1.

**Note:** In newer Linux kernels, PEAK has been integrated into the kernel properly and will follow the default SocketCAN practices.

### slcan

SerialCAN for Windows, covering devices like VScom's USB-CAN Plus.

Channels are COM ports shown in the Device Manager.

**Note:** Serial in Windows has significant performance issues, so it should be used with only a small number of sensors.

### socketcan

The default for Linux, covering most CAN and SerialCAN devices. Some PEAK devices might also work under this setting.

Channels are actual *can* and *slcan* buses in the format of *canN* or *slcanN*.

**Note:** In the newer kernel, SerialCAN is integrated into the kernel and will use the *canN* naming convention.

### sim

A simulated CAN device attempts to match the maximum number of CAN messages that can be expected from any sensors.

If the channel is "sequential", the output will have a sequence based on the computer time, starting every 15 seconds. It affects only X and Y values; Z is random. The sequence is as follows:

1. 4 seconds of tiny changes (wiggly mode)
2. 2-second intervals of 4 diagonal directions each (centre-to-corner movement)
3. 3 seconds of complete randomness

With any other value on the channel parameter, it will produce random values for X, Y and Z.

Note: This behaviour might change between releases.

## Special sensors

### Standard and Individually calibrated sensors

#### Prerequisites

- Software version 1.7.5 or newer
- Fully calibrated sensors with serial numbers or sensors supporting the calibration
- Calibration and definition files

Sensor compatibility:

| Sensor model | Standard calibration* <sup>1</sup> | Individual calibration<br>(extra file needed) |
|--------------|------------------------------------|-----------------------------------------------|
| uSPa44       | ✓                                  | ✓                                             |
| uSPa46       | ✓* <sup>2</sup>                    | ✓                                             |
| uSPa22       | ✓* <sup>3</sup>                    | ✓                                             |

File compatibility:

| Software version | .xeladef | .xdef           | .xela | .xcal |
|------------------|----------|-----------------|-------|-------|
| 1.7.5            | ✓        | ✗* <sup>4</sup> | ✗     | ✓     |
| 1.7.6+           | ✓        | ✓               | ✓     | ✓     |

*Generic parameters for uSPa44 are included with the software 1.7.6+*

*.xcal file is not required if using 1.7.6+ with .xela file unless the user needs to adjust the parameters themselves*

<sup>1</sup> For sensors getting standard calibration after software release, can be manually added with an additional file. 1.7.6 release includes standard calibration only for uSPa44

<sup>2</sup> Added to 1.7.7 build 161703

<sup>3</sup> Added to 1.7.7 build 161703

<sup>4</sup> Some .xdef files might work in 1.7.5 release, but are not recommended

## Setup

1. Copy the provided files (.xela, .xcal) to the following directory:
  - a. /etc/xela/sensors directory (Linux)
  - b. the same directory you have (or launch) the application in (Windows/Linux)
2. Edit the INI file to contain the following info:
  - a. `model = S/N` (only for fully calibrated sensors), otherwise `model = model_name`  
 Examples: `model = XC166EC61` for fully calibrated sensor and `model = uSPa44` for standard calibration on uSPa44
  - b. `calibration = on` (only required for standard calibration)

## Notes

### Comparing Sensor Calibration: Generic vs. Individual

In this software manual, we explore two calibration variants of the uSPa44 (XR2244) sensor: Generic Calibration and Individual Calibration. They both have the same purpose but with some significant differences.

#### Generic Calibration:

Generic calibration is a standardised approach, where parameters are calculated based on data collected from multiple sensors of the same type. These parameters are designed to provide a baseline level of performance suitable for various applications.

Key points to note about Generic Calibration:

**Simplicity:** Generic calibration ensures a simple activation method by writing a single line in the INI file, compared to changing every model name to a serial number and managing additional files.

**Immutable Parameters:** Generic calibration parameters are set during manufacturing and remain fixed. They are not user-adjustable and cannot be accessed from the software interface. This means they are invisible to the user and cannot be modified.



## Individual Calibration:

On the other hand, individual calibration is a highly tailored approach, explicitly performed on the specific sensor, the uSPa44 (XR2244) sensor in this example. These parameters are fine-tuned to the unique characteristics of this particular sensor.

Here's what sets Individual Calibration apart:

**Precision:** Individual calibration offers superior precision and accuracy. It optimises the sensor's performance to its full potential, ensuring the most precise measurements.

**Customizability:** One critical advantage of Individual Calibration is its flexibility. Users can modify and fine-tune parameters to suit their application's specific requirements. This means you can adapt the sensor to various tasks or conditions, providing greater control and flexibility. To receive the tailored .xcal file, please let us know. The provided .xela file also includes the parameters, but those are immutable. .xcal file exposes them and overrides the .xela file ones.

**External Access:** Individual calibration parameters are accessible from the software interface, allowing users to make real-time adjustments. This added functionality empowers users to fine-tune the sensor's behaviour on the fly.

## Forces Up to 5N:

It's essential to note that the Generic and Individual Calibration methods have been calculated for forces up to 5N at each sensing point. This ensures your sensor maintains its precision and reliability within this specified force range.

For all calibrations, you must manually edit the xServ.ini file.

For generic parameters (like with uSPa44 sensors), you need to add `calibration = on` under the `[sensor]` or `[sensorN]` section to enable the output

For individual parameters, you need to change the model to the corresponding serial number.

I.e. instead of `model = uSPa44` you will write `model = XC166EC61`

## Data

All calibrated data is available as part of the sensor message. Please see an example below with the information highlighted:

```
{'message': 786, 'time': 1619157045.6195483, 'sensors': 2, '1': {'data': 'FA00,...,FC00',
'sensor': '1', 'taxels': 4, 'model': 'uSPa22', 'calibrated': null}, '2': {'data': 'FB01,...,FC01',
'sensor': 2, 'taxels': 16, 'model': 'uSPa44', 'calibrated': [0.059,0.021,0.15,...]}}
```

For more info about the data format, please see the paragraph [Messages](#).

Note: The forces are in Newtons.

## How to use parameters in other software

You will get the parameters in the XCAL file for individually calibrated sensors. They are in a table, where each row is for 1 taxel.

**Note:** Their sequence is based on the sequence provided by the Server software, so using them directly from the CAN bus would require making sure the correct taxel IDs first. Refer to the table at the start of the Server to see ID allocations.

The following formula can be used to calculate the force. The parameters are given in a list of 27 elements, 9 for each axis (X, Y, and Z), in the XCAL file.

*Reminder: This is only available for individual calibration.*

Read:

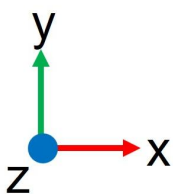
For x, p = params[0...8]

For y, p = params[9...17]

For z, p = params[18...26]

k stands for X, Y or Z

$$F_k = x_d * p_0 + y_d * p_1 + z_d * p_2 + x_d * y_d * p_3 + x_d * z_d * p_4 + y_d * z_d * p_5 + x_d^2 * p_6 + y_d^2 * p_7 + z_d^2 * p_8$$



The parameters, by default, refer to the orientation stated in the hardware manual.

When using the server application, the forces will be adjusted to display in the normal XY plane, as they do with regular measurement values, corresponding to the image shown here.

## Sensors with magnetic compensation

### Prerequisites

- Software version 1.7.6 or newer
- Sensors with magnetic compensation support
- (optional) sensor .xdef or .xela files (if not included in the software)

### Setup

- Copy the provided files (.xela or .xdef) to the following directory:
  - /etc/xela/sensors directory (Linux)
  - the same directory you have the application in (Windows)
- Edit the INI file to contain the following info:
  - `magcomp = on` under the `[sensor]` or `[sensorN]` section

### Notes

It can be provided as an individual model or as a module for the current sensor model. For example, the uSPa44 has a variant with magnetic compensation, which can be activated by the INI file with `magcomp = on` under the same sensor configuration section.

**Note:** Applying magnetic compensation for sensors without this feature can cause invalid output.

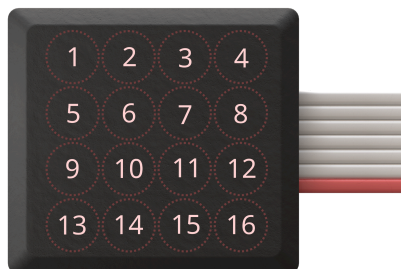
The configuration tool will recognise it as a standard sensor, and the user must change it in the INI file.

This sensor type has 2 extra settings that can be used in the INI file under the `[sensor]` and `[sensorN]` sections:

|         |                        |                                                                                                                                                              |
|---------|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rawmode | On / off (default off) | Allows to see both compensated and raw data in the XELA Viz application.<br>It also adds raw data to the message.                                            |
| magcomp | On / off (default on)  | Activates the compensation algorithm.<br>In the data stream, it will replace the original data.<br>Enable <code>rawmode</code> if you need the initial data. |

# Data format

## Sensor Layout



### Understanding Sensor Data

- **Data Reading:** The sensor data is read from left to right, starting at the top left corner and moving across each row.
- **Multiple Sensors:** If you connect multiple 4x4 sensors, the server will treat them as separate sensors. If you need a different setup, please contact us for a custom configuration.

### Determining Sensor Layout

- **Orientation:** Place the sensor flat in front of you with the cables emerging from the right side (or, if it has a connector, position the connector on the right). This is the standard orientation.
- **Layout:** The image above shows the layout for a standard 4x4 sensor. If you have a 4x6 sensor, it will have two additional columns (numbers 5 and 6 in the first row).
- **Curved Sensors:** Curved sensors will be represented as if they were flat. Always keep the cable to the right and refer to the server layout for accurate positioning.

### Server Configuration

To see the sensor layout in the server application (1.7.7 and below):

1. Locate the INI file.
2. Find the [debug] section.
3. Set `layout = full` to enable the layout display.

**Note:** The sensor is considered upright if the cables exit on the right (attachment) side.

## Messages

As the data is provided in a JSON string, there are few ways to access it, and it has the following default format:

```
{
  "message": 49090,
  "time": 1708406349.5611684,
  "version": 1.8.0b_198105,
  "sensors": 3,
  "type": "routine",
  "1": {
    "time": 1708406349.5593173,
    "sensor": "1",
    "data": "8246,81A8,8A58,...",
    "model": "XR1944",
    "taxels": 16,
    "integer": [33337, 33205, 35607, ...],
    "temp": [ 307.531, ... ],
    "ups": [105.17100960863372, ...],
    "handedness": "right",
    "calibrated": [0.059, 0.021, 0.15, ...]
  },
  "2": {
    ...
  },
  ...
}
```

## Explanations

**message** has the sequence number of the broadcast (to avoid outdated messages)

**time** has a UNIX timestamp of data creation

**sendtime** has a UNIX timestamp of publishing time

**sensors** mentions the total amount of sensors the server is handling currently

**type** mentions the message type; it can be "routine", "on-update", or "welcome"

- "routine" will show on messages sent out by the server in a specified frequency
- "on-update" will show on messages that are only sent out when a sensor update occurs
- "welcome" will show on the first message sent out by the server after connecting

*(continues on the next page)*

*(sensor message explanations continued)*

Each sensor will have its data under the element with its number, in this example, "1":

**time** shows the last update that was calculated on the sensor, relies on the CAN messages

**data** contains X, Y and Z data for all taxels (sensing points) in order, converted into HEX values (without leading '0x') as text. It means that it will need to be parsed after receiving

**integer** contains X, Y and Z data for all taxels (sensing points) in order, in standard integers (16-bit unsigned)

**sensor** has the number of the sensor in string format (redundancy check)

**taxels** has the number of the taxels meant to be sent (for confirmation only)

**model** shows the correct sensor model as a reference

**calibrated** shows the calculated forces applied to the sensor (if supported by the sensor via XCAL files and enabled in the INI file)

**temp** shows the sensor temperature for each taxel in Kelvin (can also be shown in the visualization application by enabling it in the INI file, only with sensors that have external temperature compensation)

**ups** shows the estimated update rate of the sensor for each taxel (useful with event-driven sensors to detect increased activity)

**special** contains debug data and changes from version to version. This is mainly for the internal temperature compensation (only available if server is in debug mode)

**debug\_xyz** contains debug data for calibration mechanic (only available if server is in debug mode)

Please note that this list is not exhaustive.

# Example usage

## Code

```
#!/usr/bin/env python3
import websocket
import json
from time import sleep
import threading

ip = "127.0.0.1" # server PC's IP (localhost or external IP)
port = 5000 # the port the server is running on (default is 5000)

lastmessage = {"message": "No message"} # default message you will overwrite when you get update
def on_message(wsapp, message):
    global lastmessage # globalise to overwrite the original
    try:
        data = json.loads(message)
    except Exception:
        pass
    else:
        try:
            if data["message"] == "Welcome": # get the Welcome Message with details, print if you like
                print(data)
            else:
                lastmessage = data
        except Exception:
            pass # ignore message as it's probably invalid
def threader(target, args=False, **targs):
    # args is a tuple of arguments for a threaded function; other key-value pairs will be sent to Thread
    if args:
        targs["args"]=(args,)
    thr = threading.Thread(target=target, **targs)
    thr.daemon = True
    thr.start()
def mesreader(): # this is your app reading the last valid message you received
    while True: # to run forever
        try:
            if lastmessage["message"]!="No message":
                print("I received: {}\n-----".format(str(lastmessage)))
                sleep(0.5) # your calculations and processes here (sleep is used for simulation here)
            except KeyboardInterrupt:
                break # break on KeyboardInterrupt
            except Exception as e:
                print("Exception: {}: {}".format(type(e).__name__,e))
        try: # try to close the app once you press CTRL + C
            wsapp.close()
        except Exception:
            exit()
    threader(mesreader,name="Receiver") # start you main app
websocket.setdefaulttimeout(1) # you should avoid increasing it.
wsapp = websocket.WebSocketApp("ws://{}:{}".format(ip,port), on_message=on_message) # set up WebSockets
wsapp.run_forever() # Run until the connection dies
exit()
```

## Common errors

| Error                                                                                                                   | Reason                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Anti-virus software sees the applications as potential malware or a virus                                               | Due to Python (the language in which the entire application suite is written) being an interpreted language, most AV software cannot differentiate between malware and regular applications that use sockets. To avoid this, we would need to rewrite the entire application suite in another language, which is also planned for the 2.0.0 release.                                                                                      |
| XELA_Conf found the sensor, and XELA_Server is working with no errors, but there is no XELA_Viz visual output (Windows) | The AV software may interfere with the software's networking, preventing the Visualizer from connecting to the Server. Turn off the AV software and try again                                                                                                                                                                                                                                                                             |
| XELA_Conf is giving:<br>Message not sent: OSError:<br>Transmit operation timed out                                      | Make sure the power USB is connected to a power source and the DSUB9 is firmly attached to the CAN-USB converter                                                                                                                                                                                                                                                                                                                          |
| Could not start CAN:<br>OSError: [Errno 19] No such device                                                              | Make sure to pull up the network with one of the following commands if using Linux:<br><pre>user@localhost:~\$ sudo ip link set up can0 type can bitrate 1000000</pre> or<br><pre>user@localhost:~\$ sudo slcand -o -s8 -t hw -S 3000000 /dev/ttyUSB0</pre> <pre>user@localhost:~\$ sudo ifconfig slcan0 up</pre> In the case of Windows, make sure the adapter is connected and using the specified channel (esd channel might not be 0) |
| The program is not responding to CTRL+C                                                                                 | There are times when the apps may not respond correctly to CTRL+C. In those cases, you can try the following:<br><b>Linux:</b> Ctrl-Shift-\ or pkill -9 xela_server (or whichever function is not responding)<br><b>Windows:</b> CTRL+Pause_Break, or you can simply close the app                                                                                                                                                        |
| Error connecting to CAN:<br>IOError:[Errno 19]<br>No such device                                                        | No CAN device was found. Make sure your CAN-USB device is connected, drivers installed for it, accessible for all users (pulled up) and set in the configuration file correctly ( <a href="#">see /etc/xela/xServ.ini or xServ.ini</a> )                                                                                                                                                                                                  |
| Error writing config file:<br>IOError: [Errno 2]<br>No such file or directory:<br>'/etc/xela/xServ.ini'                 | Ensure there is a /etc/xela folder (in Linux or an executable folder in Windows) and that it has 777 permissions                                                                                                                                                                                                                                                                                                                          |

|                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unable to install Python 3.7 or newer on Ubuntu            | To install different Python versions on Ubuntu, use the deadsnakes repository:<br><code>sudo add-apt-repository ppa:deadsnakes/ppa</code>                                                                                                                                                                                                                                                                                                                                                                         |
| XELA_Viz fails to launch<br>Showing ImportError            | Possible reasons are that your computer software or hardware is incompatible with SDL2 libraries inside the executable. We recommend using a different computer for visualisation in such a case, preferably running Ubuntu 20.04 (similar happens when running Ubuntu 16.04 app in newer Ubuntu release. Check release with <code>uname -a</code> )<br>A similar error on Windows means missing <a href="#">Visual C++ redistributable 2015-2019 x64</a> . Please install it, restart the computer and try again |
| The new CAN-USB converter is not working with the software | Please make sure the following steps: <ul style="list-style-type: none"> <li>• CAN-USB device is supported in the OS you are using</li> <li>• You have selected the correct bus type in the configuration file (socketcan/esd/pcan)</li> <li>• You have selected the correct channel in the configuration file (can0/slcan0/COM3, etc.)</li> <li>• Make sure the network is pulled up (Linux only)</li> <li>• ctrl_id matches with the hardware</li> </ul>                                                        |

If you find errors not listed in this file, please send an email regarding it to [info@xelarobotics.com](mailto:info@xelarobotics.com)

**Do not forget to attach files from the /etc/xela/LOG directory in Linux or the LOG directory in Windows (if using the default log directory),** a description of the failure, and additionally, you can add the terminal log (if log directory is empty, as a text, not image). If you have made your own code causing errors, please attach it.

## Known issues

### Visualisation

- Sometimes, it fails to reconnect to the server (improvements made)

### Server

- Crash, if the esd Net Bus ID is wrong or the esd CAN-USB/2 is not connected (limitation of the current ntcn library, Windows only)
- Some Linux machines report a single message per second when sensors are not connected (under investigation)

### Configurator

- Only supporting uSPa 44/46 sensors (application will be replaced in 1.8.1)

## How to report issues (checklist)

Before contacting [info@xelarobotics.com](mailto:info@xelarobotics.com) for assistance, please complete the following steps:

- Check the terminal for error messages relating to missing files or permissions. (Take screenshots or save output to a text file)
- Confirm that the sensors are correctly connected to the PC and powered on. (Both USB cables connected)
- Thoroughly review the manual to confirm that no steps were missed (e.g., server still in simulation mode).
- Include all relevant log files and terminal output as text files in your email. (Especially xela\_server.log)
- Describe the steps you have already taken to try to resolve the issue.

Please note that we may request additional information, such as images, videos, and an overview of the file tree, to resolve your issue.